

Curso Superior de Análise e Desenvolvimento de Sistemas

Especificação dos Requisitos do Software e Análise do Projeto TrainUp

Versão 1.0

Equipe:

Ana Carolina Gonçalves

Bárbara Yasmin Pimenta dos Santos Tobias

Luana Gabrielle Ferreira Guedes Paes

Pedro Augusto Lombardi da Costa

Ruan Monteiro Brito

Prof.º Wesley Fioreze

Taubaté, 2026

Histórico das Revisões

Data	Versão	Descrição	Autor
23/03/2026	1.0	Criação inicial do documento conforme entregáveis da Sprint 1	Ana, Bárbara, Luana, Pedro, Ruan

Sumário

Versão 1.0.....	1
1 Introdução	4
1.1 Objetivos deste documento	5
1.2 Escopo do produto	5
1.2.1 Nome do produto e de seus componentes principais	6
1.2.2 Descrição do produto	6
1.2.3 Missão do produto.....	6
1.2.4 Tabela de viabilidade	6
1.2.5 Justificativa	7
1.3 Definições e siglas	7
1.4 Técnica(s) utilizada(s) para levantamento de requisitos	7
2 Requisitos Específicos.....	8
3 Cronograma do Projeto.....	24
4 Protótipo do produto	24
5 Diagramas UML.....	26
5.2 Diagrama de Classes	28
5.3 Diagrama de Sequência	31
5.3.1 Diagrama de Sequência — Registrar Evolução do Aluno [RF08].....	31
5.3.2 Diagrama de Sequência — Solicitar e Confirmar Agendamento [RF14 / RF15].....	32
5.4 Diagrama de Atividade.....	33
5.4.1 Diagrama de Atividade [RF 01] – Nome do Requisito.....	33
5.4.2 Diagrama de Atividade [RF 01] – Nome do Requisito.....	33
6. Casos de Testes.....	33
7. Apêndices.....	33

1 Introdução

Este documento apresenta a Especificação de Requisitos de Software (ERS) do TrainUp, solução desenvolvida pela equipe para a demanda “Serviços para Personal Trainer”, pesquisada e selecionada como ativa na Plataforma SAGA SENAI de Inovação. O problema identificado é a dificuldade de personal trainers autônomos e pequenos estúdios em administrar alunos, treinos e evolução física de forma manual (planilhas, papel ou aplicativos de mensagens), o que gera retrabalho, perda de histórico e dificuldade para gerar relatórios de evolução e organizar a agenda de atendimentos. O TrainUp propõe um MVP (Produto Mínimo Viável) que permite ao Personal Trainer cadastrar alunos, montar treinos, registrar a evolução física e de desempenho e gerar relatórios automáticos a partir de dados reais, enquanto o Aluno acompanha sua evolução e solicita agendamentos. Este documento formaliza os requisitos funcionais, não funcionais, regras de negócio, a modelagem e o protótipo que orientam o desenvolvimento das Sprints 2 e 3 do projeto.

1.1 Objetivos deste documento

Este documento visa formalizar e detalhar de forma técnica e inequívoca todos os requisitos funcionais, não funcionais e restrições de projeto do sistema TrainUp. O objetivo principal é estabelecer um contrato técnico entre a equipe de desenvolvimento e os stakeholders, garantindo que as necessidades de negócio sejam traduzidas fielmente em funcionalidades de software, minimizando ambiguidades e riscos durante o ciclo de vida de desenvolvimento (SDLC).

Público Alvo: Stakeholders do projeto, analistas de sistemas, desenvolvedores de software, equipe de QA (Quality Assurance) e gestores de projeto.

1.2 Escopo do produto

O projeto TrainUp consiste no desenvolvimento de uma solução tecnológica voltada para a gestão de atendimentos de personal trainers e o acompanhamento da evolução física e de desempenho de seus alunos. O sistema abrange desde a camada de persistência de dados (PostgreSQL) até interfaces web responsivas para personal e aluno, integrando conceitos de Ciência de Dados (cálculo de indicadores de evolução), Automação Industrial (pipeline de coleta/processamento de dados de treino, com rastreabilidade de origem manual/simulada) e Computação em Nuvem (hospedagem e configuração via variáveis de ambiente).

1.2.1 Nome do produto e de seus componentes principais

- TrainUp: núcleo do sistema.
- Módulo de Alunos e Treinos: cadastro de alunos e montagem de treinos com exercícios.
- Módulo de Evolução e Relatórios (Ciência de Dados): registro histórico de evolução e cálculo de indicadores (analytics_service).
- Módulo de Agenda (Serviços e Agendamentos): definição de disponibilidade e solicitação/confirmação de atendimentos.
- API REST (FastAPI): camada de regras de negócio e autenticação JWT.

1.2.2 Descrição do produto

O TrainUp é um sistema web para personal trainers organizarem o trabalho com seus alunos. De forma simples, é como uma “ficha de acompanhamento digital” que substitui planilhas e anotações soltas.

De forma técnica, trata-se de uma aplicação cliente-servidor com front-end em React consumindo uma API REST em FastAPI, que aplica regras de negócio e persiste os dados em PostgreSQL, com camadas de repositório e serviço separando acesso a dados e lógica de domínio (Programação Orientada a Objetos). O benefício central é transformar registros manuais e dispersos de evolução física em indicadores confiáveis e relatórios automáticos, apoiando a tomada de decisão do personal trainer e a motivação do aluno.

1.2.3 Missão do produto

Prover uma solução digital eficiente, segura e responsiva para o acompanhamento de alunos de personal trainers, promovendo organização, rastreabilidade dos dados de evolução e melhoria na comunicação entre profissional e aluno.

1.2.4 Tabela de viabilidade

Item	Descrição	Valor Unitário
Hospedagem	Servidor cloud mensal	R\$ 30,00
Domínio	Registro anual	R\$ 40,00
Desenvolvimento	Mão de obra estimada	R\$ 2.000,00
Ferramentas	IDEs e plataformas	R\$ 0,00

Valor total estimado: R\$ 2.070,00

Podem pensar em mais linhas para a tabela e diferentes modelos de precificação!

(AINDA VOU PASSAR AULA DE PRECIFICAÇÃO PARA VOCÊS)

1.2.5 Justificativa

O valor considera custos operacionais mínimos e mão de obra técnica. O produto possui potencial de escalabilidade, podendo ser comercializado como SaaS.

1.3 Definições e siglas

Número	Sigla	Definição
1	ERS	Especificação de Requisitos de Software.
2	RF / RNF	Requisito Funcional / Requisito Não Funcional.
3	RN	Regra de Negócio: Normas e diretrizes que regem a lógica do domínio.
4	CRUD	Create, Read, Update, Delete (Operações básicas de banco de dados).
5	API	Application Programming Interface.
6	JWT	JSON Web Token — padrão usado para autenticação e transporte de sessão do usuário.
7	MVP	Produto Mínimo Viável.
8	RN (dado)	Campo origem do registro de evolução: manual (inserido pelo pessoal) ou simulado (dado de demonstração), nunca apresentado como coletado de equipamento real.

1.4 Técnica(s) utilizada(s) para levantamento de requisitos

As seguintes técnicas foram utilizadas:

- Análise da demanda publicada na Plataforma SAGA SENAI de Inovação

- Entrevistas informais com personal trainers autônomos sobre a rotina de acompanhamento de alunos
- Observação de processos manuais (planilhas e aplicativos de mensagens)
- Análise de sistemas similares de gestão de estúdios/academias

Os registros encontram-se documentados no Documento Inicial da Demanda (Entregável 1), incluindo a evidência de captura de tela da demanda ativa na Plataforma SAGA SENAI.

2 Requisitos Específicos

2.1 Requisitos Funcionais (RF)

A lista abaixo apresenta a priorização dos 16 requisitos funcionais do TrainUp (Entregável 3). Em seguida, cada requisito é detalhado com a estrutura de fluxos (Entregável 4): identificador, descrição, ator, prioridade, gatilho, pré-condições, fluxo principal numerado, subfluxos/fluxos alternativos e dependências.

ID	Nome	Prioridade	No MVP?
RF01	Registrar-se como Personal Trainer	Alta	Sim
RF02	Registrar-se como Aluno (vínculo a um Personal)	Alta	Sim
RF03	Autenticar-se (Login)	Alta	Sim
RF04	Cadastrar Aluno (pelo Personal)	Alta	Sim
RF05	Editar dados do Aluno	Média	Sim
RF06	Criar Treino com Exercícios	Alta	Sim
RF07	Consultar Treinos	Alta	Sim
RF08	Registrar Evolução do Aluno	Crítica (núcleo)	Sim
RF09	Consultar Histórico e Indicadores de Evolução	Crítica (núcleo)	Sim
RF10	Gerar Relatório de Evolução	Crítica (núcleo)	Sim
RF11	Consultar Relatórios (Aluno/Personal)	Alta	Sim
RF12	Cadastrar e Gerenciar Serviços	Média	Sim (Prioridade 2)
RF13	Definir Disponibilidade de Agenda	Média	Sim (Prioridade 2)
RF14	Solicitar Agendamento	Média	Sim (Prioridade 2)

RF15	Confirmar / Rejeitar / Cancelar Agendamento	Média	Sim (Prioridade 2)
RF16	Consultar Dashboard (resumo)	Baixa	Desejável

[RF01] — Registrar-se como Personal Trainer

- **Descrição:** O sistema deve permitir que um novo Personal Trainer crie sua conta, informando dados de perfil profissional.
- **Ator principal:** Visitante (não autenticado).
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Usuário acessa a tela “Criar conta de personal”.
- **Pré-condições:** Nenhuma (usuário não autenticado).
- **Fluxo Principal:**
 1. O ator acessa o formulário de registro de Personal Trainer.
 2. O sistema apresenta os campos: nome, e-mail, senha, bio (opcional), especialidade (opcional).
 3. O ator preenche os campos e confirma.
 4. O sistema valida os dados e verifica se o e-mail já está cadastrado (RN08).
 5. O sistema cria o registro Usuario (tipo PERSONAL) e o perfil PersonalTrainer associado.
 6. O sistema gera um token JWT e autentica o ator automaticamente.
 7. O sistema redireciona para o Dashboard do Personal.
- **Subfluxos / Fluxos Alternativos:**
 - A1 — E-mail já cadastrado:** o sistema impede a criação, retorna erro 409 e exibe mensagem “E-mail já cadastrado”.
 - A2 — Campos obrigatórios vazios:** o sistema impede o envio e destaca os campos pendentes.
- **Dependências:** Nenhuma. Origem: demanda de acesso ao sistema.

[RF02] — Registrar-se como Aluno

- **Descrição:** O sistema deve permitir que um novo Aluno crie sua conta e, opcionalmente, já se vincule a um Personal Trainer existente.
- **Ator principal:** Visitante (não autenticado).
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Usuário acessa a tela “Criar conta de aluno”.

- **Pré-condições:** Nenhuma.
- **Fluxo Principal:**
 1. O ator acessa o formulário de registro de Aluno.
 2. O sistema apresenta os campos: nome, e-mail, senha, telefone (opcional), objetivo (opcional) e a lista de Personal Trainers disponíveis (endpoint público GET /api/personal-trainers).
 3. O ator preenche os campos, opcionalmente seleciona um Personal, e confirma.
 4. O sistema valida a unicidade do e-mail (RN08) e, se informado, a existência do Personal selecionado.
 5. O sistema cria o Usuario (tipo ALUNO) e o perfil Aluno vinculado ao Personal escolhido (RN01).
 6. O sistema gera um token JWT e autentica o ator.
 7. O sistema redireciona para o Dashboard do Aluno.
- **Subfluxos / Fluxos Alternativos:**
 - A1 — E-mail já cadastrado:** erro 409, com mensagem correspondente.
 - A2 — Personal Trainer informado não existe:** erro 404 “Personal trainer não encontrado”.
- **Dependências:** RF01 (lista de personal trainers cadastrados).

[RF03] — Autenticar-se (Login)

- **Descrição:** O sistema deve autenticar Personal ou Aluno por e-mail e senha, retornando um token de sessão.
- **Ator principal:** Personal Trainer ou Aluno.
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Usuário acessa a tela de login e envia as credenciais.
- **Pré-condições:** Possuir conta cadastrada (RF01 ou RF02).
- **Fluxo Principal:**
 1. O ator informa e-mail e senha.
 2. O sistema localiza o usuário pelo e-mail.
 3. O sistema verifica a senha (hash com passlib).
 4. O sistema identifica o perfil (personal_trainer ou aluno) associado ao usuário.
 5. O sistema gera um token JWT contendo perfil_id, tipo, nome e email.
 6. O sistema redireciona o ator ao dashboard correspondente ao seu tipo.
- **Subfluxos / Fluxos Alternativos:**

A1 — Credenciais inválidas: erro 401 “Credenciais inválidas”, sem indicar se o problema foi o e-mail ou a senha (segurança).

- **Dependências:** RF01, RF02.

[RF04] — Cadastrar Aluno (pelo Personal)

- **Descrição:** O Personal Trainer cadastra um novo Aluno diretamente vinculado a si (fluxo alternativo ao autocadastro do Aluno).
- **Ator principal:** Personal Trainer autenticado.
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Personal acessa “Gerenciar alunos” → “Novo aluno”.
- **Pré-condições:** Personal autenticado (RF03).
- **Fluxo Principal:**
 1. O Personal solicita a inclusão de um novo aluno.
 2. O sistema apresenta o formulário (nome, e-mail, senha inicial, telefone, objetivo, data de nascimento).
 3. O Personal preenche e confirma.
 4. O sistema valida a unicidade do e-mail (RN08).
 5. O sistema cria o Usuario (tipo ALUNO) e o Aluno já vinculado ao personal_trainer_id do ator (RN01).
 6. O sistema exibe o aluno na lista de alunos do Personal.
- **Subfluxos / Fluxos Alternativos:**
 - A1 — E-mail já cadastrado:** erro 409.
 - A2 — Campos obrigatórios ausentes:** a validação de formulário impede o envio.
- **Dependências:** RF03.

[RF05] — Editar dados do Aluno

- **Descrição:** O Personal Trainer atualiza dados cadastrais de um aluno vinculado a ele.
- **Ator principal:** Personal Trainer autenticado.
- **Prioridade / MVP:** Média / Sim (MVP).
- **Gatilho:** Personal acessa a ficha do aluno → “Editar”.
- **Pré-condições:** O aluno deve estar vinculado ao Personal autenticado (RN02).
- **Fluxo Principal:**

1. O Personal seleciona um aluno da lista.
 2. O sistema exibe os dados atuais em formulário editável.
 3. O Personal altera os campos desejados (telefone, objetivo, dados gerais) e confirma.
 4. O sistema valida o vínculo (RN02) e persiste as alterações.
 5. O sistema exibe a confirmação.
- **Subfluxos / Fluxos Alternativos:**
 - A1 — Aluno não vinculado ao Personal:** erro 403 “Você não pode alterar este aluno”.
 - **Dependências:** RF04.

[RF06] — Criar Treino com Exercícios

- **Descrição:** O Personal monta um treino para um aluno, composto por uma lista ordenada de exercícios com séries, repetições e carga.
- **Ator principal:** Personal Trainer autenticado.
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Personal acessa “Treinos” → “Novo treino”.
- **Pré-condições:** Aluno vinculado ao Personal (RN02).
- **Fluxo Principal:**
 1. O Personal seleciona o aluno destinatário e informa o nome/observações do treino.
 2. O sistema apresenta a lista de exercícios cadastrados (catálogo reutilizável) e permite buscar ou criar um novo exercício por nome.
 3. O Personal adiciona um ou mais exercícios, definindo séries, repetições, carga e ordem.
 4. O Personal confirma a criação do treino.
 5. O sistema valida o vínculo do aluno (RN02), persiste o treino e cada item TreinoExercicio associado.
 6. O sistema exibe o treino criado.
- **Subfluxos / Fluxos Alternativos:**
 - A1 — Exercício informado por nome não existe no catálogo:** o sistema cria automaticamente um novo Exercicio (get_or_create_exercicio).

A2 — Nem exercicio_id nem exercicio_nome informados: erro 422

“Informe exercicio_id ou exercicio_nome”.

A3 — Aluno não vinculado ao Personal: erro 403.

- **Dependências:** RF04.

[RF07] — Consultar Treinos

- **Descrição:** Personal e Aluno consultam a lista de treinos de um aluno específico.
- **Ator principal:** Personal Trainer ou Aluno autenticado.
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Acesso à tela “Meus treinos” (aluno) ou “Treinos do aluno” (personal).
- **Pré-condições:** RN02 — o aluno só vê os próprios treinos; o personal só vê treinos de alunos vinculados.
- **Fluxo Principal:**
 1. O ator acessa a listagem de treinos.
 2. O sistema valida o acesso (RN02).
 3. O sistema retorna os treinos do aluno, cada um com seus exercícios ordenados.
 4. O ator pode abrir um treino específico para ver os detalhes completos.
- **Subfluxos / Fluxos Alternativos:**
 - A1 — Acesso não autorizado:** erro 403.
 - A2 — Nenhum treino cadastrado:** o sistema exibe estado vazio (“Nenhum treino cadastrado ainda”).
- **Dependências:** RF06.

[RF08] — Registrar Evolução do Aluno (núcleo do MVP)

- **Descrição:** O Personal registra um novo ponto de acompanhamento da evolução física e de desempenho do aluno (peso, altura, medidas, carga de referência, repetições, desempenho, frequência semanal).
- **Ator principal:** Personal Trainer autenticado.
- **Prioridade / MVP:** Crítica / Sim (núcleo do MVP).
- **Gatilho:** Personal acessa a ficha do aluno → “Registrar evolução”.
- **Pré-condições:** Aluno vinculado ao Personal (RN02, RN07).
- **Fluxo Principal:**
 1. O Personal seleciona o aluno e a data do registro.

2. O sistema apresenta o formulário: peso (kg), altura (cm), medidas (JSON opcional), carga de referência (kg), repetições, desempenho (escala 0-10), frequência semanal, observações e origem do dado (manual/simulado).
 3. O Personal preenche e confirma.
 4. O sistema valida o vínculo (RN02) e persiste um novo registro imutável de Evolucao, associado à data informada (RN04).
 5. O sistema atualiza automaticamente os gráficos e indicadores exibidos (RF09).
- **Subfluxos / Fluxos Alternativos:**
 - A1 — Aluno não vinculado ao Personal:** erro 403 “Você não tem permissão para este aluno”.
 - A2 — Nenhum campo numérico preenchido:** o sistema permite salvar apenas com observações, mas os indicadores que dependem do campo ausente não são recalculados para esse ponto.
 - **Dependências:** RF04.

[RF09] — Consultar Histórico e Indicadores de Evolução (núcleo do MVP)

- **Descrição:** O sistema calcula, a partir do histórico real de registros, indicadores agregados (variação de peso/carga em valor e percentual, desempenho médio, frequência média semanal) e séries temporais para gráficos.
- **Ator principal:** Personal Trainer ou Aluno.
- **Prioridade / MVP:** Crítica / Sim (núcleo do MVP).
- **Gatilho:** Acesso à tela “Minha evolução” (aluno) ou à ficha do aluno (personal).
- **Pré-condições:** RN02.
- **Fluxo Principal:**
 1. O ator acessa a tela de evolução do aluno.
 2. O sistema valida o acesso (RN02) e busca todos os registros de Evolucao do aluno, ordenados por data.
 3. O sistema calcula (camada analytics_service): peso inicial/atual e variação; carga inicial/atual e variação; desempenho médio; frequência média semanal; e as séries temporais de peso, carga e desempenho.
 4. O sistema apresenta os indicadores em cartões numéricos e os gráficos de evolução (Recharts).
- **Subfluxos / Fluxos Alternativos:**

A1 — Nenhum registro de evolução: o sistema exibe estado vazio (“Nenhum registro ainda”).

A2 — Acesso não autorizado: erro 403.

- **Dependências:** RF08.

[RF10] — Gerar Relatório de Evolução (núcleo do MVP)

- **Descrição:** O Personal gera um relatório consolidado de evolução do aluno para um período (data início/fim), com resumo textual gerado automaticamente a partir dos indicadores reais.
- **Ator principal:** Personal Trainer autenticado.
- **Prioridade / MVP:** Crítica / Sim (núcleo do MVP).
- **Gatilho:** Personal acessa “Relatórios” → “Gerar relatório”.
- **Pré-condições:** Aluno vinculado ao Personal (RN02).
- **Fluxo Principal:**
 1. O Personal seleciona o aluno e o período (data início e fim).
 2. O sistema valida o acesso (RN02) e busca os registros de evolução dentro do período.
 3. O sistema calcula os indicadores do período (mesma lógica do RF09).
 4. O sistema gera um resumo textual automático (por exemplo: “5 registro(s) de evolução no período. Redução de peso de 2.30 kg (-3.1%). Aumento de carga de referência de 5.00 kg (12.5%). Desempenho médio registrado: 8.2/10.”).
 5. O sistema persiste o Relatório com o snapshot dos indicadores (JSON) e o resumo.
 6. O sistema exibe o relatório gerado.
- **Subfluxos / Fluxos Alternativos:**
 - A1 — Nenhum registro no período:** o resumo automático informa “Nenhum registro de evolução encontrado no período selecionado.”
 - A2 — O Personal informa uma observação manual:** ela substitui o resumo automático.
- **Dependências:** RF08, RF09.

[RF11] — Consultar Relatórios

- **Descrição:** Aluno e Personal consultam a lista de relatórios já gerados e o detalhe de cada um.
- **Ator principal:** Personal Trainer ou Aluno.
- **Prioridade / MVP:** Alta / Sim (MVP).
- **Gatilho:** Acesso à tela “Relatórios”.
- **Pré-condições:** RN02.
- **Fluxo Principal:**
 1. O ator acessa a lista de relatórios do aluno.
 2. O sistema valida o acesso e retorna os relatórios ordenados por período.
 3. O ator abre um relatório específico para ver o resumo e os indicadores completos.
- **Subfluxos / Fluxos Alternativos:**
 - A1 — Relatório não encontrado:** erro 404.
 - A2 — Acesso não autorizado:** erro 403.
- **Dependências:** RF10.

[RF12] — Cadastrar e Gerenciar Serviços

- **Descrição:** O Personal cadastra os serviços que oferece (ex.: “Avaliação física”, “Sessão avulsa”), com duração e valor.
- **Ator principal:** Personal Trainer autenticado.
- **Prioridade / MVP:** Média / Sim (Prioridade 2).
- **Gatilho:** Personal acessa “Serviços” → “Novo serviço”.
- **Pré-condições:** Personal autenticado.
- **Fluxo Principal:**
 1. O Personal informa nome, descrição, duração em minutos e valor do serviço.
 2. O sistema persiste o serviço vinculado ao Personal, ativo por padrão.
 3. O Personal pode editar ou inativar (ativo = false) um serviço existente.
 4. O Aluno visualiza a lista de serviços ativos de um Personal ao solicitar agendamento (RF14).
- **Subfluxos / Fluxos Alternativos:**
 - A1 — Edição de serviço de outro Personal:** bloqueada — o endpoint só permite atualizar serviços do próprio Personal autenticado.
- **Dependências:** Nenhuma.

[RF13] — Definir Disponibilidade de Agenda

- **Descrição:** O Personal define janelas de horário (dia da semana, hora início, hora fim) em que aceita atendimentos.
- **Ator principal:** Personal Trainer autenticado.
- **Prioridade / MVP:** Média / Sim (Prioridade 2).
- **Gatilho:** Personal acessa “Agenda” → “Definir disponibilidade”.
- **Pré-condições:** Personal autenticado.
- **Fluxo Principal:**
 1. O Personal seleciona o dia da semana e o intervalo de horário.
 2. O sistema persiste a Disponibilidade vinculada ao Personal.
 3. O Aluno consulta a disponibilidade pública do Personal ao solicitar um agendamento (RF14).
- **Subfluxos / Fluxos Alternativos:**
 - Sem fluxos alternativos críticos** identificados nesta versão do MVP.
- **Dependências:** Nenhuma.

[RF14] — Solicitar Agendamento

- **Descrição:** O Aluno solicita um atendimento com um Personal em um horário específico, opcionalmente vinculado a um serviço.
- **Ator principal:** Aluno autenticado.
- **Prioridade / MVP:** Média / Sim (Prioridade 2).
- **Gatilho:** Aluno acessa “Agenda” → escolhe horário disponível → “Solicitar”.
- **Pré-condições:** Aluno autenticado; Personal e (opcionalmente) serviço existentes.
- **Fluxo Principal:**
 1. O Aluno consulta a disponibilidade e os serviços do Personal.
 2. O Aluno escolhe data/hora de início e, opcionalmente, um serviço (que define a duração).
 3. O sistema calcula a data/hora de fim (duração do serviço ou 60 minutos padrão).
 4. O sistema cria o Agendamento com status PENDENTE.
 5. O sistema notifica visualmente o Personal (lista de pendentes no dashboard).
- **Subfluxos / Fluxos Alternativos:**

A1 — Serviço informado não pertence ao Personal escolhido: erro 404

“Serviço não encontrado para este personal”.

A2 — Múltiplas solicitações pendentes no mesmo horário: permitido —
cabe ao Personal decidir qual confirmar (RN03).

- **Dependências:** RF12, RF13.

[RF15] — Confirmar / Rejeitar / Cancelar Agendamento

- **Descrição:** O Personal confirma ou rejeita solicitações pendentes; Personal ou Aluno podem cancelar um agendamento próprio.
- **Ator principal:** Personal Trainer (confirmar/rejeitar) e Aluno ou Personal (cancelar).
- **Prioridade / MVP:** Média / Sim (Prioridade 2).
- **Gatilho:** Personal abre a lista de agendamentos pendentes; ou Aluno/Personal aciona “Cancelar” em um agendamento existente.
- **Pré-condições:** O agendamento deve pertencer ao ator (RN06); para confirmação, o ator deve ser o Personal do agendamento.
- **Fluxo Principal:**
 1. O Personal seleciona um agendamento PENDENTE.
 2. O sistema verifica se já existe outro agendamento CONFIRMADO no mesmo intervalo de horário para este Personal (RN03).
 3. Se não houver conflito, o sistema altera o status para CONFIRMADO.
 4. O sistema atualiza a visualização da agenda de ambos os atores.
- **Subfluxos / Fluxos Alternativos:**
 - A1 — Conflito de horário (RN03):** o sistema rejeita a confirmação com erro 409 “Já existe um agendamento confirmado neste horário”.
 - A2 — Rejeição:** o Personal altera o status para REJEITADO (mesmo fluxo de atualização de status, sem verificação de conflito).
 - A3 — Cancelamento:** o dono (Aluno ou Personal) altera o status para CANCELADO; outro ator que não seja o dono recebe erro 403.
- **Dependências:** RF14.

[RF16] — Consultar Dashboard (agregação, não crítico)

- **Descrição:** Tela inicial que resume, em um só lugar, indicadores já calculados por outros RFs (totais de alunos, treinos e agendamentos pendentes para o Personal; indicadores de evolução e próximos agendamentos para o Aluno).

- **Ator principal:** Personal Trainer ou Aluno.
- **Prioridade / MVP:** Baixa / Desejável.
- **Gatilho:** Acesso à tela inicial (“Painel”) após o login.
- **Pré-condições:** RN02.
- **Fluxo Principal:**
 1. O sistema agrega, em uma única resposta, dados já existentes dos módulos de alunos (RF04), treinos (RF06), evolução (RF09) e agenda (RF14/RF15).
 2. O sistema não introduz nova regra de negócio — apenas consolida indicadores já calculados.
- **Subfluxos / Fluxos Alternativos:**

Sem fluxos alternativos próprios: eventuais erros herdam o tratamento dos RFs de origem dos dados agregados.
- **Dependências:** RF04, RF06, RF09, RF14, RF15.

2.2 Requisitos Não Funcionais (RNF)

Estipulam as qualidades e restrições técnicas do sistema. A tabela a seguir prioriza os 7 requisitos não funcionais do TrainUp (Entregável 3); em seguida, cada um é detalhado como um cenário de verificação (Entregável 5), com categoria, componente afetado, condições iniciais, procedimento de verificação, métrica, resultado esperado e evidência a coletar.

ID	Categoria	Prioridade
RNF01	Segurança (autenticação/senha)	Alta
RNF02	Confidencialidade / controle de acesso	Crítica
RNF03	Tempo de resposta	Alta
RNF04	Usabilidade / Responsividade	Alta
RNF05	Disponibilidade (hospedagem em nuvem)	Média
RNF06	Confiabilidade dos dados (imutabilidade da evolução)	Alta
RNF07	Manutenibilidade (arquitetura em camadas)	Média

[RNF01] — Segurança de autenticação

- **Categoria:** Segurança
- **Descrição:** Senhas nunca são armazenadas em texto puro; toda rota autenticada exige um JWT válido.

- **Componente / processo afetado:** app/core/security.py, deps.py (back-end).
- **Condições iniciais:** Ambiente de demonstração com banco populado (seed_data.py).
- **Procedimento de verificação:** (1) Inspeccionar a coluna senha_hash no banco e confirmar que não é texto plano; (2) chamar um endpoint protegido (ex.: GET /api/students) sem header Authorization e depois com um token inválido.
- **Métrica / parâmetro:** Hash com passlib (bcrypt); resposta 401 sem token válido.
- **Resultado esperado:** Senha sempre em hash; requisição sem token válido retorna 401 Unauthorized.
- **Evidência a coletar:** Print do valor do campo no banco + print da resposta 401 no Swagger (/docs) ou Postman.

[RNF02] — Confidencialidade / controle de acesso por perfil

- **Categoria:** Segurança / Confidencialidade
- **Descrição:** Um Aluno nunca acessa dados de outro Aluno; um Personal nunca acessa alunos não vinculados a ele (RN02).
- **Componente / processo afetado:**
StudentService.garantir_acesso_aluno_por_perfil, usado por TrainingService, EvolutionService e ReportService.
- **Condições iniciais:** Dois alunos de demonstração distintos, autenticados com tokens diferentes.
- **Procedimento de verificação:** Autenticar como Aluno A e tentar consultar GET /api/evolution/aluno/{id_do_aluno_B}.
- **Métrica / parâmetro:** Resposta HTTP 403 Forbidden.
- **Resultado esperado:** Acesso negado, com mensagem explícita.
- **Evidência a coletar:** Print da resposta 403 no Swagger/Postman.

[RNF03] — Tempo de resposta

- **Categoria:** Desempenho
- **Descrição:** Em ambiente de demonstração, a consulta de indicadores de evolução (RF09) deve responder em até 3 segundos.
- **Componente / processo afetado:** GET /api/evolution/aluno/{id}/indicadores.
- **Condições iniciais:** Banco populado pelo seed_data.py (aproximadamente 6 meses de histórico por aluno).

- **Procedimento de verificação:** Medir o tempo de resposta da rota (aba Network do navegador ou comando `time curl ...`) em 5 chamadas consecutivas.
- **Métrica / parâmetro:** Tempo de resposta ≤ 3 segundos.
- **Resultado esperado:** Todas as 5 chamadas dentro do limite.
- **Evidência a coletar:** Print da aba Network do navegador com os tempos, ou saída do comando `time`.

[RNF04] — Usabilidade e responsividade

- **Categoria:** Usabilidade
- **Descrição:** A interface deve ser utilizável em telas de desktop e mobile, com mensagens claras de erro/sucesso.
- **Componente / processo afetado:** Front-end React + Tailwind (todas as páginas).
- **Condições iniciais:** Aplicação rodando em localhost:5173.
- **Procedimento de verificação:** Abrir o dashboard do Personal e do Aluno em resolução desktop (1440px) e mobile (375px, DevTools “Toggle device toolbar”); provocar um erro de validação (ex.: login com senha errada) e observar a mensagem exibida.
- **Métrica / parâmetro:** Nenhum elemento cortado/sobreposto nas duas resoluções; mensagem de erro visível e compreensível.
- **Resultado esperado:** Layout adaptado (grid responsivo) e mensagem de erro clara (“Credenciais inválidas”).
- **Evidência a coletar:** Prints das duas resoluções + print da mensagem de erro.

[RNF05] — Disponibilidade (Computação em Nuvem)

- **Categoria:** Disponibilidade / Computação em Nuvem
- **Descrição:** O sistema deve estar preparado para implantação em ambiente de computação em nuvem, utilizando configurações por meio de variáveis de ambiente e sem armazenamento de credenciais ou informações sensíveis diretamente no código-fonte.
- **Componente / processo afetado:** Backend e configurações de infraestrutura.

- **Condições iniciais:** Repositório do projeto publicado no GitHub e aplicação preparada para implantação em ambiente de nuvem.
- **Procedimento de verificação:** Inspecionar o código-fonte e confirmar a ausência de strings de conexão, credenciais ou segredos fixos. Verificar também se as configurações necessárias para execução em ambiente de nuvem estão definidas por variáveis de ambiente.
- **Métrica / parâmetro:** Nenhuma credencial ou informação sensível hardcoded no código-fonte; aplicação apta a ser executada em ambiente de computação em nuvem.
- **Resultado esperado:** Sistema disponível em ambiente de computação em nuvem, permitindo o acesso ao front-end e a comunicação com o back-end de forma funcional.
- **Evidência a coletar:** Print ou registro das configurações de ambiente utilizadas, juntamente com a comprovação da aplicação em execução no ambiente de nuvem, quando disponível.

[RNF06] — Confiabilidade dos dados de evolução (imutabilidade)

- **Categoria:** Confiabilidade
- **Descrição:** Um novo registro de evolução nunca deve sobrescrever um registro anterior — cada data gera uma nova linha no histórico (RN04).
- **Componente / processo afetado:** EvolutionService.registrar_evolucao, tabela evolucoes.
- **Condições iniciais:** Aluno com ao menos um registro de evolução existente.
- **Procedimento de verificação:** Registrar uma nova evolução para o mesmo aluno em data diferente e consultar o histórico completo (GET /api/evolution/aluno/{id}).
- **Métrica / parâmetro:** Quantidade de registros no histórico aumenta em +1; o registro anterior permanece inalterado.
- **Resultado esperado:** Histórico com todos os registros anteriores intactos e o novo adicionado.
- **Evidência a coletar:** Print do histórico antes e depois da inclusão.

[RNF07] — Manutenibilidade (arquitetura em camadas)

- **Categoria:** Manutenibilidade
- **Descrição:** O back-end deve separar claramente rotas (API), regras de negócio (services) e acesso a dados (repositories), evidenciando POO.
- **Componente / processo afetado:** Toda a pasta backend/app (api/, services/, repositories/, models/).
- **Condições iniciais:** Código-fonte disponível no repositório.
- **Procedimento de verificação:** Inspecionar um fluxo completo (ex.: RF08) e confirmar que a rota (api/evolution.py) não contém lógica de negócio, apenas delega ao EvolutionService, que por sua vez usa TrackingRepository para acesso a dados.
- **Métrica / parâmetro:** Zero lógica de negócio/consulta SQL direta dentro dos arquivos de api/.
- **Resultado esperado:** Separação de responsabilidades confirmada por leitura de código.
- **Evidência a coletar:** Print/trecho dos três arquivos (rota, service, repository) lado a lado.

2.3 Regras de Negócio (RN)

As Regras de Negócio definem as restrições lógicas e os processos que o software deve obedecer obrigatoriamente. São 8 regras priorizadas para o MVP do TrainUp (Entregável 3):

- **RN01 — Vínculo Aluno–Personal:** Um Aluno está vinculado a no máximo um Personal Trainer por vez. (Prioridade: Alta.)
- **RN02 — Controle de acesso por perfil:** Um Aluno só acessa os próprios dados; um Personal só acessa alunos vinculados a ele. (Prioridade: Crítica.)
- **RN03 — Conflito de agendamento confirmado:** Não é permitido existir dois agendamentos CONFIRMADOS no mesmo horário para o mesmo Personal (solicitações PENDENTES concorrentes são permitidas). (Prioridade: Alta.)
- **RN04 — Imutabilidade da evolução:** Todo registro de Evolução é imutável e possui data própria — nunca sobrescreve um registro anterior. (Prioridade: Alta.)
- **RN05 — Origem do dado de evolução:** Todo registro de Evolução deve indicar sua origem (manual ou simulado), nunca apresentado como dado de equipamento real. (Prioridade: Média.)
- **RN06 — Dono do agendamento:** Um agendamento só pode ser cancelado pelo seu dono (o Aluno que solicitou ou o Personal responsável). (Prioridade: Alta.)

- **RN07 — Responsabilidade sobre a evolução:** Apenas o Personal Trainer responsável pode registrar evolução e criar treinos para um Aluno. (Prioridade: Alta.)
- **RN08 — Unicidade de e-mail:** O e-mail de cadastro é único no sistema, tanto para Personal quanto para Aluno. (Prioridade: Média.)

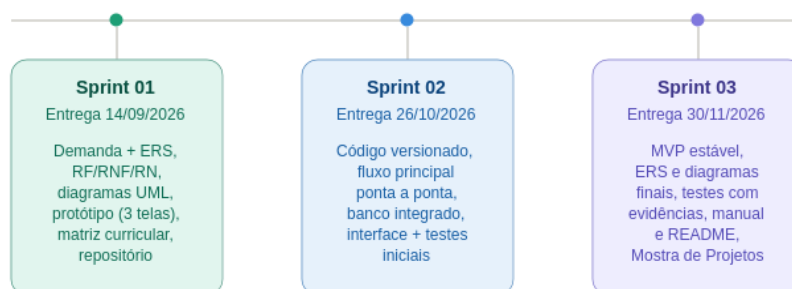
3 Cronograma do Projeto

ID	Entregável	Referência no Guia	Data sugerida	Responsável	Apoio (opcional)	Status	% Concluído	Prazo final	Observações
1	Documento inicial da demanda (com evidência SAGA ativa)	Guia — Seção 2 (Entregável 1)	10/08/2026	Todos		Concluído	100%	14/09/2026	
2	ERS — Especificação de Requisitos de Software (consolidação final)	Guia — Seção 3 (Entregável 2)	14/09/2026	Bárbara		Concluído	0	14/09/2026	
3	Lista priorizada de RF, RNF e RN	Guia — Seção 4 (Entregável 3)	17/08/2026	Bárbara		Concluído	0	14/09/2026	
4	Especificação textual dos fluxos, subfluxos e exceções dos RFs	Guia — Seção 5 (Entregável 4)	24/08/2026	Luana		Concluído	0	14/09/2026	
5	Cenários textuais de verificação dos RNFs	Guia — Seção 6 (Entregável 5)	24/08/2026	Ruan		Concluído	100%	14/09/2026	
6	Diagrama de Casos de Uso	Guia — Seção 7 (Entregável 6)	31/08/2026	Ruan		Concluído	100%	14/09/2026	
7	Diagrama de Classes	Guia — Seção 8 (Entregável 7)	31/08/2026	Pedro		Concluído	100%	14/09/2026	
8	Diagrama Entidade-Relacionamento (DER) + dicionário de dados	Guia — Seção 9 (Entregável 8)	31/08/2026	Pedro		Concluído	0	14/09/2026	
9	Dois diagramas de sequência dos processos principais	Guia — Seção 10 (Entregável 9)	31/08/2026	Luana		Concluído	0	14/09/2026	
10	Protótipo das 3 principais interfaces e navegação	Guia — Seção 11 (Entregável 10)	31/08/2026	Ana		Concluído	100%	14/09/2026	
11	Matriz de integração das unidades curriculares	Guia — Seção 12 (Entregável 11)	17/08/2026	Ana		Concluído	0	14/09/2026	
12	Repositório com README e estrutura inicial organizada	Guia — Seção 13 (Entregável 12)	03/08/2026	Pedro		Concluído	0	14/09/2026	

3.1 Planejamento de Sprints

A execução seguirá a metodologia ágil **Scrum**, dividida em ciclos de **<X>** semanas.

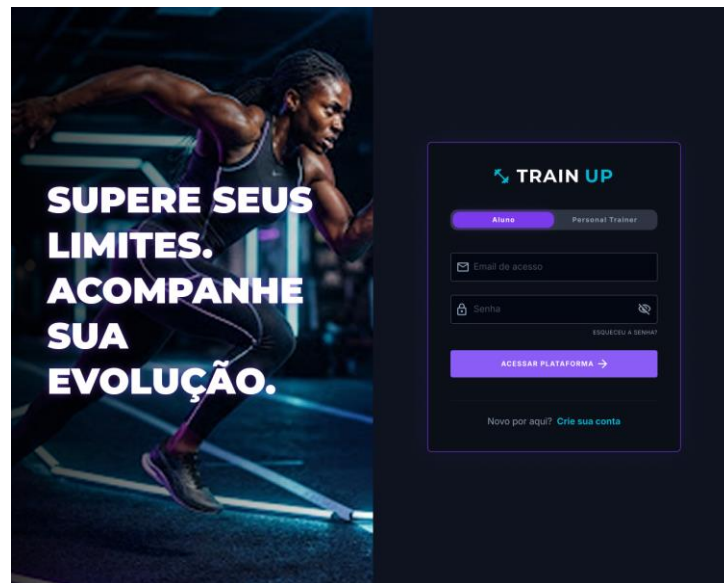
- **Sprint 01:** Levantamento de requisitos e modelagem de dados.
- **Sprint 02:** Desenvolvimento do Módulo de Autenticação e RF01 a RF03.
- **Sprint 03:** Implementação das Regras de Negócio complexas e Testes de Integração.



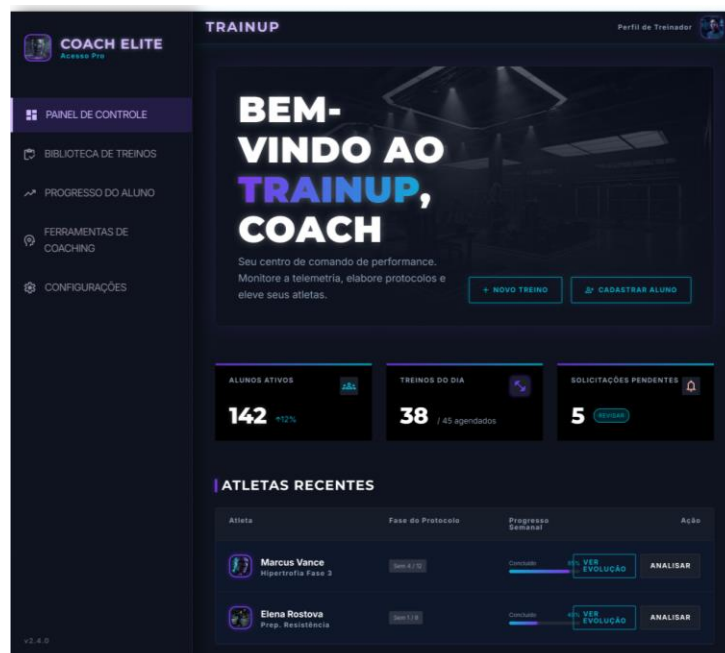
4 Protótipo do produto

Nesta seção, apresentam-se as interfaces de alta fidelidade que guiaram o desenvolvimento do front-end. Como o TrainUp já possui front-end funcional (React + Tailwind CSS).

- **Protótipo:** representado pelas telas reais do sistema implementado, capturadas em docs/prototipo/ (login, dashboard do Personal Trainer e tela de evolução do Aluno).
- **Principais Telas:**
 1. Tela 1 — Login (/login): fundo escuro (tema dark), cartão central com campos de e-mail e senha, botão “Entrar”, alternância entre acesso de Aluno e de Personal Trainer, e links para criação de conta. Erro de credencial exibe faixa de aviso acima do formulário.



2. Tela 2 — Dashboard do Personal Trainer (/personal): painel de controle com indicadores (alunos ativos, treinos do dia, solicitações pendentes), lista de “Atletas recentes” com a fase de treinamento e a evolução semanal de cada aluno, e atalhos para novo treino e cadastro de aluno.



3. Tela 3 — Minha Evolução (Aluno) (/aluno/evolucao): cartões de indicador (peso corporal, carga média, frequência semanal) com variação percentual, e o gráfico “Curva de Evolução” sobrepondo as séries de carga e peso ao longo do tempo, com exportação do relatório em PDF.



- **Fluxo de navegação entre as telas:**

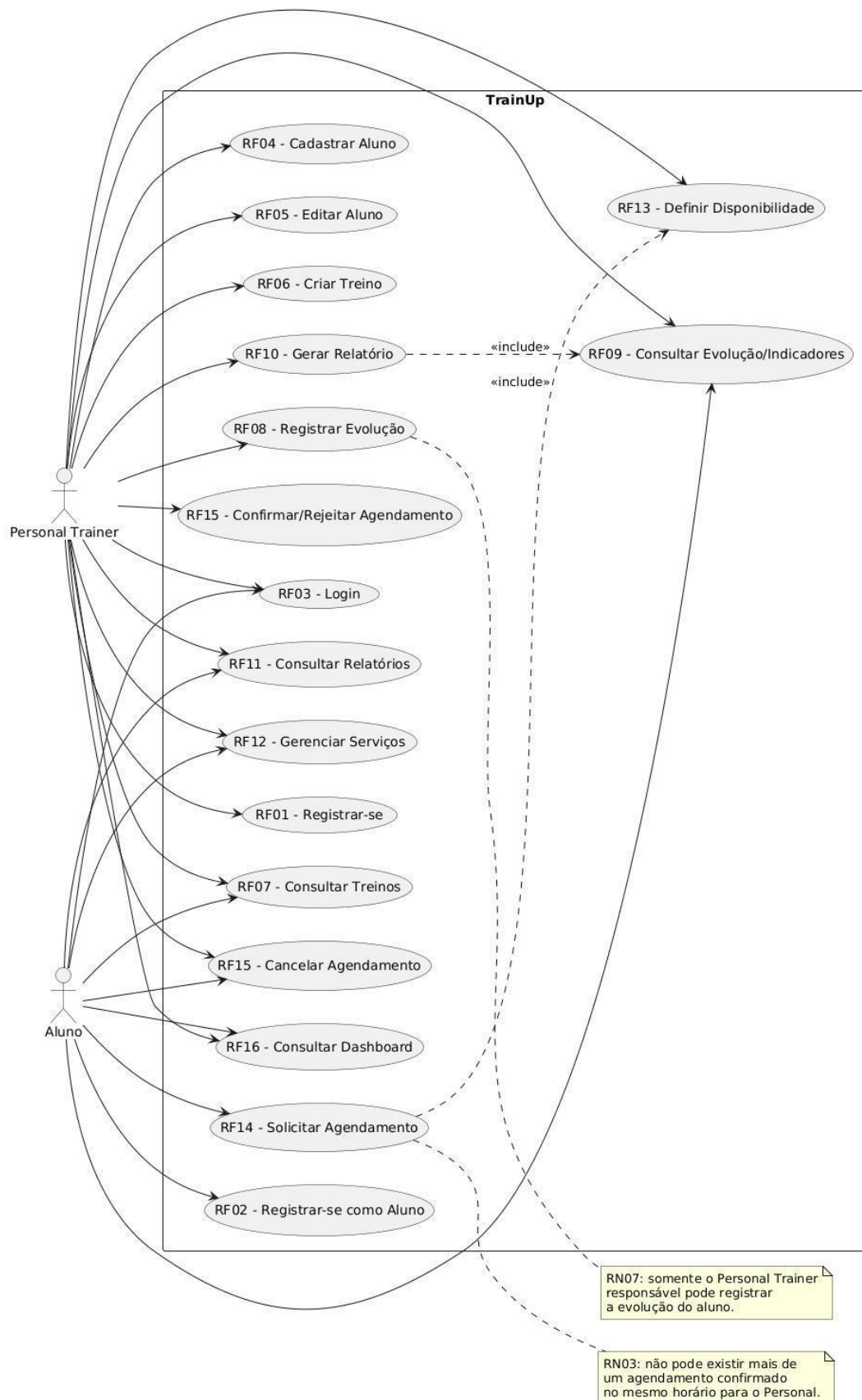
1. Login com credenciais de Personal Trainer → Dashboard do Personal Trainer (/personal).
2. Login com credenciais de Aluno → Dashboard do Aluno → Minha Evolução (/aluno/evolucao), acessível pelo menu lateral.
3. No Dashboard do Personal, o atalho “Ver agenda” leva à Agenda do Personal (/personal/agenda).

5 Diagramas UML

5.1 Diagrama de Casos de Uso

O diagrama a seguir representa os dois atores do TrainUp — Personal Trainer e Aluno — e os 16 casos de uso correspondentes aos requisitos funcionais RF01 a RF16. Destacam-se duas relações

«include»: “Solicitar Agendamento” inclui “Definir Disponibilidade” (o aluno sempre consulta a disponibilidade antes de solicitar) e “Gerar Relatório” inclui “Consultar Evolução/Indicadores” (o relatório sempre reaproveita o cálculo de indicadores). Duas notas destacam as regras de negócio mais relevantes: RN07 (somente o Personal Trainer responsável pode registrar a evolução do aluno) e RN03 (não pode existir mais de um agendamento confirmado no mesmo horário para o mesmo Personal). Login e cadastro existem apenas para viabilizar o fluxo central de evolução → relatório, que é o núcleo de valor da solução.



5.2 Diagrama de Classes

O diagrama de classes evidencia a Programação Orientada a Objetos e o domínio do TrainUp, com 11 classes principais (Usuario, PersonalTrainer, Aluno, Exercicio, Treino, TreinoExercicio, Evolucao, Relatorio, Servico, Disponibilidade e Agendamento) e três enumerações (TipoUsuario, OrigemDado e StatusAgendamento). Usuario possui exatamente um perfil (PersonalTrainer ou Aluno). Um PersonalTrainer orienta vários Alunos (RN01), cria Treinos e registra Evolucoes; Treino compõe-se de vários TreinoExercicio, que referenciam o catálogo reutilizável de Exercicios. As notas de regra de negócio destacam a imutabilidade da Evolucao (RN04) e a restrição de conflito de horário no Agendamento (RN03).

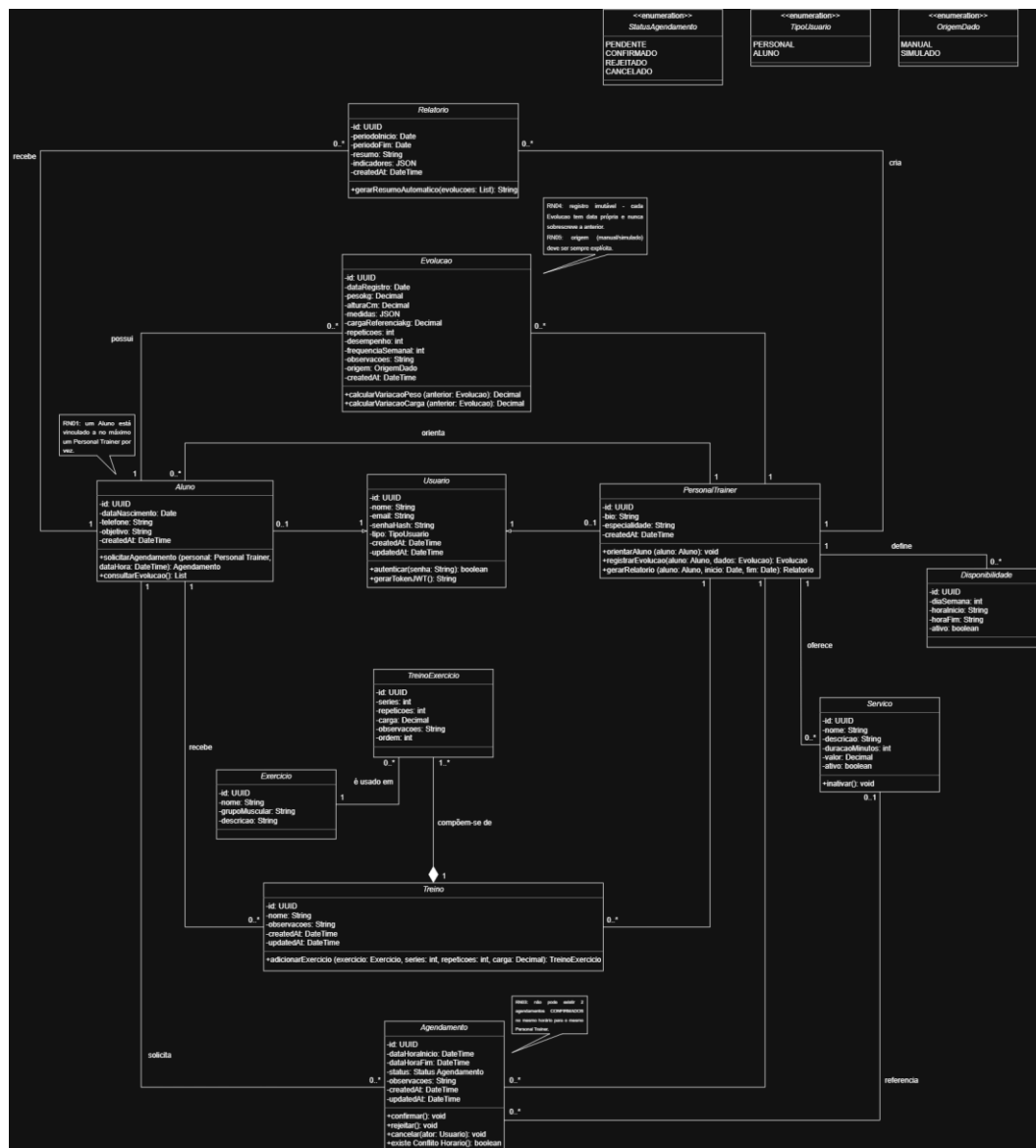
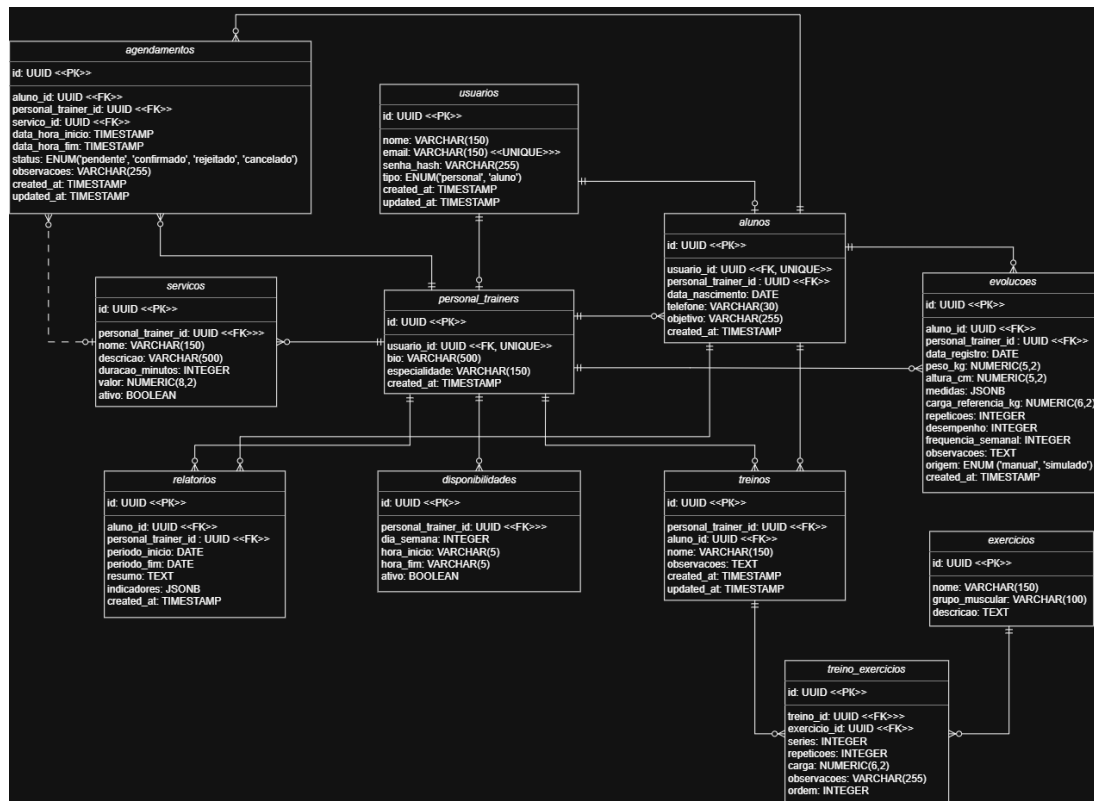


Diagrama Entidade-Relacionamento (DER) e Dicionário de Dados

O banco de dados PostgreSQL do TrainUp é composto por 11 tabelas. Os relacionamentos usuarios→personal_trainers e usuarios→alunos são 1:1 (um usuário possui exatamente um perfil). personal_trainers→alunos é 1:N (RN01). O relacionamento servicos→agendamentos é opcional (0..1), pois um

agendamento pode não referenciar nenhum serviço específico (duração padrão de 60 minutos). As demais associações seguem cardinalidade 1:N a partir de `personal_trainers` e `alunos` em direção a `treinos`, `evolucoes`, `relatorios`, `disponibilidades` e `agendamentos`.



Dicionário de dados — tabela evolucoes (núcleo do sistema):

Coluna	Tipo	Obrigatório	Descrição
id	UUID	Sim (PK)	Identificador único do registro
aluno_id	UUID	Sim (FK)	Aluno ao qual o registro pertence
personal_trainer_id	UUID	Sim (FK)	Personal que registrou o dado
data_registro	DATE	Sim	Data do ponto de acompanhamento
peso kg	NUMERIC(5,2)	Não	Peso do aluno na data
altura cm	NUMERIC(5,2)	Não	Altura do aluno
medidas	JSONB	Não	Medidas corporais livres (ex.: cintura, braço)
carga_referencia_kg	NUMERIC(6,2)	Não	Carga de referência do exercício acompanhado
repeticoes	INTEGER	Não	Repetições realizadas na carga de referência
desempenho	INTEGER	Não	Nota de desempenho (escala 0-10)
frequencia_semanal	INTEGER	Não	Treinos realizados na

			semana
origem	ENUM	Sim	manual ou simulado (RN05)
created_at	TIMESTAMP	Sim	Data/hora de criação do registro

Dicionário de dados — tabela agendamentos:

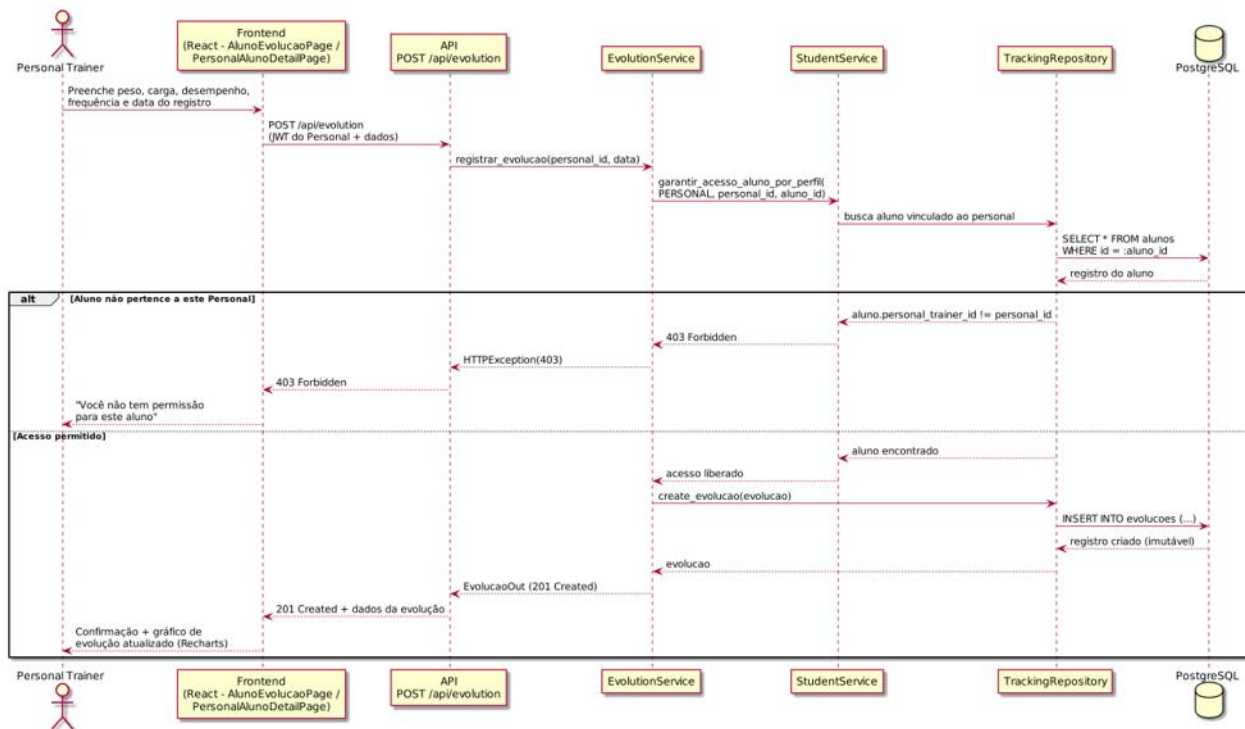
Coluna	Tipo	Obrigatório	Descrição
id	UUID	Sim (PK)	Identificador único
aluno_id	UUID	Sim (FK)	Aluno que solicitou
personal_trainer_id	UUID	Sim (FK)	Personal responsável
servico_id	UUID	Não (FK)	Serviço vinculado (opcional)
data_hora_inicio	TIMESTAMP	Sim	Início do atendimento
data_hora_fim	TIMESTAMP	Sim	Fim (calculado pela duração do serviço)
status	ENUM	Sim	pendente, confirmado, rejeitado, cancelado (RN03)
observacoes	VARCHAR(255)	Não	Observações do aluno

O dicionário completo das 11 tabelas (todas as colunas) está disponível no repositório em docs/diagramas/dicionario_dados.docx

5.3 Diagrama de Sequência

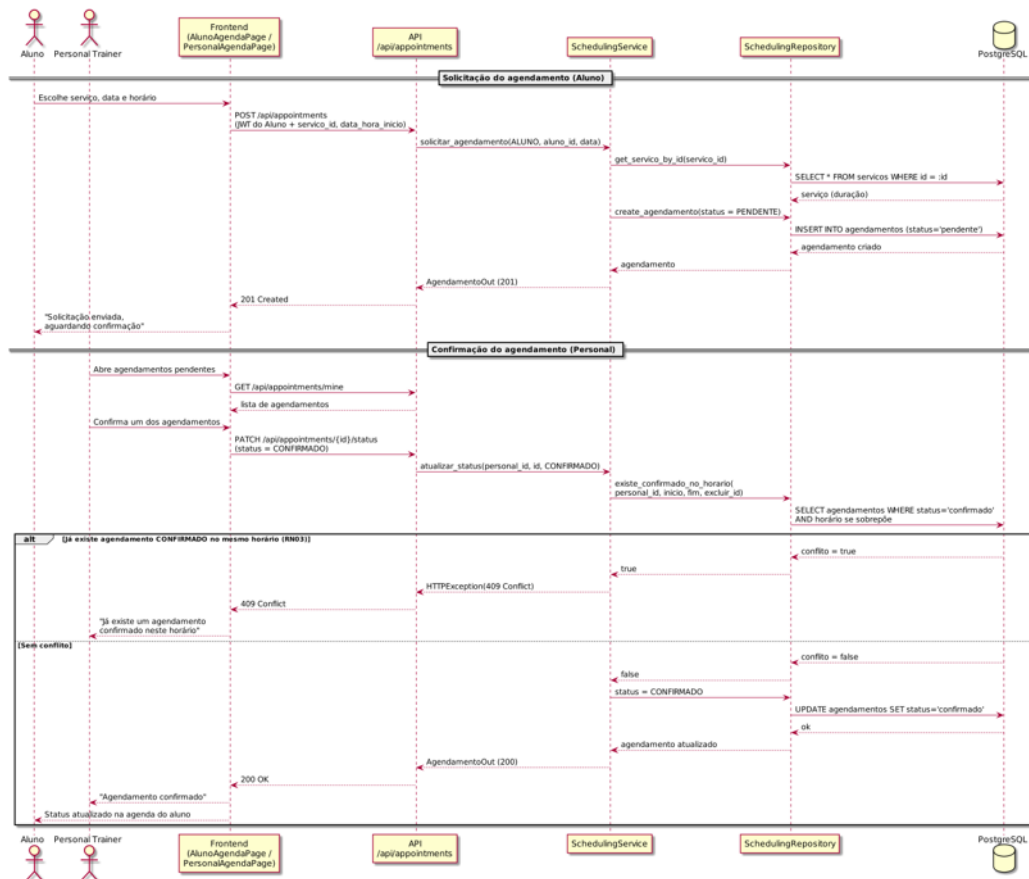
5.3.1 Diagrama de Sequência — Registrar Evolução do Aluno [RF08]

Fluxo: o Personal Trainer preenche peso, carga, desempenho, frequência e a data do registro. O Frontend chama `POST /api/evolution`; a API delega a `EvolutionService.registrar_evolucao`, que chama `StudentService.garantir_acesso_aluno_por_perfil` para validar o vínculo aluno–personal (RN02/RN07) junto ao `TrackingRepository`. Em bloco alt: se o aluno não pertence a este Personal, retorna 403 Forbidden; caso o acesso seja permitido, o sistema grava o novo registro imutável (INSERT em `evolucoes`) e retorna 201 Created com o gráfico de evolução atualizado (Recharts).



5.3.2 Diagrama de Sequência — Solicitar e Confirmar Agendamento [RF14 / RF15]

Fluxo em duas seções: (1) Solicitação — o Aluno escolhe serviço, data e horário; o Frontend chama POST /api/appointments; o SchedulingService consulta a duração do serviço e cria o Agendamento com status PENDENTE (201 Created). (2) Confirmação — o Personal Trainer abre os agendamentos pendentes e confirma um deles via PATCH /api/appointments/{id}/status; antes de confirmar, o SchedulingService verifica existe_confirmado_no_horario(...) no repositório. Em bloco alt: se já existir um agendamento CONFIRMADO no mesmo horário (RN03), retorna 409 Conflict; caso não haja conflito, o status muda para CONFIRMADO e a agenda de ambos os atores é atualizada.



5.4 Diagrama de Atividade

5.4.1 Diagrama de Atividade [RF 01] – Nome do Requisito

5.4.2 Diagrama de Atividade [RF 01] – Nome do Requisito

6. Casos de Testes

CT001 – Cadastro de usuário válido

CT0022 – Cadastro de Médico válido

7. Apêndices